

Modulo 6 — Funzionalità native

Camera • Galleria • Geolocalizzazione • Permessi • Sensori • Push (Expo + Firebase/FCM) • Reanimated

Obiettivi del modulo

Cosa imparerai

- Usare API di device in modo affidabile: permessi, errori, fallback.
- Distinguere Expo (managed) e React Native CLI (bare) e sapere cosa cambia.
- Implementare: picker galleria, fotocamera, geolocalizzazione, sensori, notifiche.
- Capire “cosa succede sotto” (JS ↔ nativo) con un modello mentale semplice.

Mappa del modulo

Argomenti

- 1) Modello mentale: JS, Metro, runtime, moduli nativi.
- 2) Expo vs CLI: differenze pratiche.
- 3) Permessi: pattern, stati, fallback UX.
- 4) Camera + galleria (Expo) + note su CLI.
- 5) Geolocalizzazione (una volta, watch, background: concetti e trade-off).
- 6) Sensori + UI.
- 7) Notifiche: locali e push (Expo) + panoramica Firebase/FCM.
- 8) Reanimated: animazioni fluide.
- 9) Problemi comuni + checklist.

1) Cosa succede “sotto”

JS ↔ Native, in parole semplici

Modello mentale: due mondi

JS e Nativo

- Il codice React/React Native (JS/TS) gira in un “runtime JavaScript” dentro l'app.
- La UI vera (bottoni, liste, testo) è nativa: iOS/Android la disegnano davvero.
- Per usare funzioni di sistema (camera, GPS, notifiche) RN chiama “moduli nativi”.

Pipeline in sviluppo (Expo/Metro)

Step-by-step

- Scrivi JS/TS + JSX.
- Babel/TypeScript trasformano il codice in JavaScript compatibile.
- Metro crea un bundle (JS + assets) e lo serve via dev server.
- Il device carica il bundle e lo esegue: Fast Refresh aggiorna senza rebuild completa.

Pipeline in produzione

Step-by-step

- Metro genera un bundle “prod” (ottimizzato).
- Durante la build nativa (Xcode/Gradle o EAS) il bundle viene impacchettato nell'app.
- L'app installata contiene già il bundle: non serve dev server.

JS Runtime ↔ Native

Bridge / JSI (idea)

- Quando premi un bottone: l'evento nasce nel mondo nativo.
- RN lo passa al codice JS (handler onPress).
- Quando chiedi la camera: JS chiama un modulo → il sistema apre la UI nativa.
- La comunicazione può passare da Bridge o JSI (più moderno).

2) Expo vs React Native CLI

Cosa cambia davvero (ma usiamo Expo)

Expo (managed)

Cosa ti dà

- API pronte (Expo Modules): camera, location, notifiche, sensori...
- Setup rapido, meno configurazione nativa.
- Build native con EAS (quando serve).
- Ottimo per la maggior parte dei progetti e per imparare.

React Native CLI (bare)

Cosa comporta

- Più controllo: puoi integrare qualsiasi SDK nativo.
- Più toolchain: Xcode, CocoaPods, Gradle, Android Studio.
- Più punti di rottura (versioni, plugin, configurazioni, signing).

Scelta pratica (approccio consigliato)

In sintesi

- Expo per iniziare e per la maggior parte dei casi d'uso.
- CLI solo se serve un SDK nativo non coperto (o vincoli enterprise).
- Expo non è un vicolo cieco: EAS + config plugins coprono molti casi.

3) Permessi

Pattern + fallback

Perché i permessi sono centrali

Idea chiave

- La feature può funzionare... ma fallire perché manca un permesso.
- Gli utenti possono negare e cambiare impostazioni in qualsiasi momento.
- Gestione corretta: permesso → uso → errori → fallback.

Stati tipici (semplificati)

Permesso

- granted: puoi usare la feature.
- denied: l'utente ha negato (a volte puoi chiedere di nuovo).
- blocked / never ask again: serve guidare l'utente alle Impostazioni.
- canceled: l'utente ha chiuso un flow (non è un errore).

Pattern consigliato

Checklist

- Spiega il valore prima della richiesta (1 riga).
- Richiedi permesso su azione utente (tasto)
- Se negato: fallback + bottone 'Apri Impostazioni'.
- Evita loop: non richiedere in continuazione.

Fallback UX (esempi)

Cosa mostrare

- Camera negata → upload da galleria o placeholder.
- Location negata → inserimento manuale città/indirizzo.
- Notifiche negate → banner in-app + guida a Impostazioni.
- Sensori non disponibili → disabilita feature e spiega.

4) Camera e Galleria

expo-image-picker + note CLI

Fotocamera vs Galleria

Cosa cambia

- Fotocamera: catturi una nuova foto → permesso Camera.
- Galleria: scegli foto esistente → permesso Media Library/Photos.
- In entrambi i casi ottieni un URI locale + metadati.

ImagePicker — galleria

Spiegazione

- Richiedi permesso Media Library.
- Apri picker di sistema (UI nativa).
- Se canceled: ritorni ok=false senza errori.
- Se ok: prendi uri/width/height.

ImagePicker — galleria

Codice completo

expo-image-picker

```
import * as ImagePicker from "expo-image-picker";

export async function pickFromLibrary() {
  const perm = await ImagePicker.requestMediaLibraryPermissionsAsync();
  if (!perm.granted) return { ok: false, reason: "permission" };

  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    quality: 0.8,
    allowsEditing: true,
  });

  if (result.canceled) return { ok: false, reason: "canceled" };

  const asset = result.assets[0];
  return { ok: true, uri: asset.uri, width: asset.width, height: asset.height };
}
```

ImagePicker — fotocamera

Spiegazione

- Richiedi permesso Camera.
- Apri camera di sistema.
- Se canceled: ritorna ok=false.
- Se ok: ritorna uri foto.

ImagePicker — fotocamera

Codice completo

Camera + Settings

```
import * as ImagePicker from "expo-image-picker";
import { Linking } from "react-native";

export async function takePhoto() {
  const perm = await ImagePicker.requestCameraPermissionsAsync();
  if (!perm.granted) return { ok: false, reason: "permission" };

  const result = await ImagePicker.launchCameraAsync({
    quality: 0.8,
    allowsEditing: false,
  });

  if (result.canceled) return { ok: false, reason: "canceled" };

  return { ok: true, uri: result.assets[0].uri };
}

export function openSettings() {
  return Linking.openSettings();
}
```

Mostrare l'immagine in UI

Suggerimenti

- Usa `<Image source={{ uri }} />` (o `expo-image` per performance).
- Gestisci loading/errore: placeholder + retry.
- Se uri è null: mostra stato vuoto (non crashare).

Preview immagine

Codice

Render Image

```
import { Image } from "react-native";

export function Preview({ uri }: { uri?: string | null }) {
  if (!uri) return null;
  return (
    <Image
      source={{ uri }}
      style={{ width: 220, height: 220, borderRadius: 16 }}
      resizeMode="cover"
    />
  );
}
```

CLI: cosa devi aggiungere in più

Nota

- In bare spesso usi librerie come react-native-image-picker.
- Serve configurazione nativa: iOS Info.plist (privacy strings) + Android permissions.
- Dipendenze native → CocoaPods/Gradle (più complesso di Expo).

5) Geolocalizzazione

expo-location + concetti

Scenari tipici

Location

- Una volta: ottieni coordinate e le mostri.
- Watch: subscription con update periodici.
- Background: tracking fuori app (solo se necessario).

Trade-off importanti

Batteria e privacy

- Più accuracy e più frequenza = più batteria.
- Spiega sempre perché serve la location (fiducia).
- Non tracciare in background se non serve davvero.

Posizione una volta

Codice

expo-location

```
import * as Location from "expo-location";

export async function getCurrentPosition() {
  const { status } = await Location.requestForegroundPermissionsAsync();
  if (status !== "granted") return { ok: false, reason: "permission" };

  const pos = await Location.getCurrentPositionAsync({
    accuracy: Location.Accuracy.Balanced,
  });

  return { ok: true, lat: pos.coords.latitude, lng: pos.coords.longitude };
}
```

Watch position

Idea

- Avvii una subscription che ti invia update periodici.
- Stop è fondamentale: rimuovi listener quando non serve.
- Fallback: se permesso negato → modalità manuale (città/indirizzo).

Watch position

Codice

watchPositionAsync

```
import * as Location from "expo-location";

export async function watchPosition(onUpdate) {
  const { status } = await Location.requestForegroundPermissionsAsync();
  if (status !== "granted") return { ok: false, reason: "permission" };

  const sub = await Location.watchPositionAsync(
    { accuracy: Location.Accuracy.Balanced, TimeInterval: 3000, distanceInterval: 10
    },
    (pos) => onUpdate(pos.coords)
  );

  return { ok: true, stop: () => sub.remove() };
}
```


Background location (concetto)

Quando serve

- Solo per casi come tracking consegne/sport/navigation.
- Richiede permessi extra e comunicazione trasparente all'utente.
- In Expo si usano TaskManager + `startLocationUpdatesAsync` (avanzato).

CLI: differenze importanti

Nota

- In bare potresti dover usare Foreground Service su Android.
- iOS: background modes e regole più rigide.
- Expo semplifica parte del setup ma concetti e limiti rimangono.

6) Sensori

expo-sensors

Accelerometro e giroscopio

Cosa misurano

- Accelerometro: accelerazione (movimento/inclinazione).
- Giroscopio: rotazione.
- Dati continui → attenzione a frequenza update (batteria).

Sensori — base

Codice

expo-sensors

```
import { Accelerometer, Gyroscope } from "expo-sensors";

export function startAccelerometer(onData) {
  Accelerometer.setUpdateInterval(200);
  const sub = Accelerometer.addListener(onData);
  return () => sub.remove();
}

export function startGyroscope(onData) {
  Gyroscope.setUpdateInterval(200);
  const sub = Gyroscope.addListener(onData);
  return () => sub.remove();
}
```

Sensori in UI

Pattern

- Memorizza l'ultimo valore in state.
- Mostra valori o usali per muovere elementi UI.
- Rimuovi listener su unmount per evitare leak.

Sensori — UI

Esempio

Hook + UI

```
import { useEffect, useState } from "react";
import { Text, View } from "react-native";
import { startAccelerometer } from "../sensors";

export function AccelViewer() {
  const [data, setData] = useState({ x: 0, y: 0, z: 0 });

  useEffect(() => startAccelerometer(setData), []);

  return (
    <View style={{ padding: 16 }}>
      <Text>X: {data.x.toFixed(2)}</Text>
      <Text>Y: {data.y.toFixed(2)}</Text>
      <Text>Z: {data.z.toFixed(2)}</Text>
    </View>
  );
}
```

7) Reanimated

Animazioni fluide

Perché Reanimated

Idea

- Animazioni fluide devono lavorare vicino alla UI nativa.
- Reanimated sposta lavoro su UI thread (meno jank).
- Utile per: gesture, parallax, transizioni.

Reanimated — esempio

Codice

Reanimated (base)

```
import Animated, { useSharedValue, useAnimatedStyle, withSpring } from
"react-native-reanimated";
import { Pressable, Text } from "react-native";

export function BouncyBox() {
  const scale = useSharedValue(1);

  const style = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }],
  }));

  return (
    <Pressable onPress={() => (scale.value = withSpring(scale.value === 1 ? 1.12 :
1))}>
      <Animated.View style={[{ width: 120, height: 120, borderRadius: 24,
        backgroundColor: "#4f46e5" }, style] />
      <Text style={{ marginTop: 8 }}>Tap per animare</Text>
    </Pressable>
  );
}
```

Expo vs CLI

Setup

- Expo: spesso già configurato nei template.
- CLI: attenzione a Babel plugin e versioni compatibili.

8) Notifiche

Locali + Push + panoramica FCM

Notifiche locali vs Push

Differenza

- Locali: generate dall'app sul device.
- Push: generate da un server e consegnate da provider.

Push: concetti chiave

Token e flusso

- Token = indirizzo del device.
- L'app registra token e lo salva sul backend.
- Il backend invia al provider (Expo/FCM/APNs) → device riceve.
- Gestisci: foreground, background, tap.

Expo Push Token

Registrazione

expo-notifications

```
import * as Notifications from "expo-notifications";
import * as Device from "expo-device";

export async function registerForPush() {
  if (!Device.isDevice) return { ok: false, reason: "simulator" };

  const { status: existingStatus } = await Notifications.getPermissionsAsync();
  const finalStatus =
    existingStatus === "granted"
      ? existingStatus
      : (await Notifications.requestPermissionsAsync()).status;

  if (finalStatus !== "granted") return { ok: false, reason: "permission" };

  const token = (await Notifications.getExpoPushTokenAsync()).data;
  return { ok: true, token };
}
```

Notification handler

Foreground behavior

setNotificationHandler

```
import * as Notifications from "expo-notifications";

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: false,
    shouldSetBadge: false,
  }),
});
```


Android: Notification Channel

Perché serve

- Controlla importanza, suono e vibrazione.
- Senza canale, comportamento può essere diverso da quanto ti aspetti.

Android channel

Codice

Channel

```
import * as Notifications from "expo-notifications";

export async function ensureChannel() {
  await Notifications.setNotificationChannelAsync("default", {
    name: "Default",
    importance: Notifications.AndroidImportance.MAX,
  });
}
```

Notifica locale (per test o fallback)

Idea

- Ottima per testare UI di notifica senza server.
- Utile per reminder in-app.

Notifica locale

Codice

`scheduleNotificationAsync`

```
import * as Notifications from "expo-notifications";

export async function scheduleLocalNotification() {
  await Notifications.scheduleNotificationAsync({
    content: { title: "Ciao!", body: "Questa è una notifica locale." },
    trigger: { seconds: 5 },
  });
}
```

Firestore/FCM — cosa sapere

Panoramica

- FCM è il provider principale su Android e può integrarsi con iOS (APNs).
- Expo Push è più semplice per iniziare; FCM è per casi avanzati.
- FCM richiede progetto Firestore e configurazioni native (più vicine al mondo CLI).

Expo Push vs FCM

Quando scegliere

- Expo Push: prototipi e molti progetti standard.
- FCM: topic, integrazioni avanzate, requisiti specifici Android.
- Approccio: parti con Expo Push, poi evolvi se serve.

Fallback e casi limite

Notifiche

- Permesso negato: banner in-app + guida a Impostazioni.
- Simulator: usa notifiche locali o log (push reali spesso non funzionano).
- Token assente: retry controllato e diagnostica.

Pausa

15 minuti

Riprendiamo tra 15 minuti

Permessi

Sintomi → cause

- Feature non parte → permesso non granted.
- Prompt non appare → già negato/blocked.
- Soluzione: controllare status + openSettings + fallback.

Immagini

Sintomi → cause

- Non vedi immagine → uri errato o permessi mancanti.
- Soluzione: usare uri da ImagePicker + placeholder/errore.

Location

Sintomi → cause

- Sempre null → permesso negato o servizi disattivati.
- Update troppo frequenti → batteria alta.
- Soluzione: accuracy bilanciata + stop subscription.

Push

Sintomi → cause

- Non arriva nulla → permesso negato, token mancante, channel Android assente.
- Soluzione: verifiche step-by-step + notifiche locali per test.

Checklist finale

Prima di integrare una feature nativa

- Hai gestito permessi + stati?
- Hai fallback UX per l'utente?
- Hai logging minimo per debug?
- Hai gestito cleanup (listener/subscription)?

Domande di ripasso

Domanda 1 di 5

1. Qual è il pattern corretto per usare una feature di device in modo affidabile?

- ☐ Permesso → uso → gestione errori → fallback
- ☐ Uso diretto → se crasha, si aggiusta
- ☐ Solo permesso, poi non controlli più
- ☐ Solo in emulator

Domande di ripasso

Domanda 2 di 5

2. Perché la cancellazione nel picker (canceled) va gestita?

- ☐ Perché è un caso normale e non un errore
- ☐ Perché significa sempre bug
- ☐ Perché indica permessi mancanti
- ☐ Perché serve riavviare Metro

Domande di ripasso

Domanda 3 di 5

3. Qual è un buon fallback se l'utente nega la geolocalizzazione?

- ☐ Input manuale città/indirizzo e disabilitare feature dipendente
- ☐ Riprova a chiedere ogni secondo
- ☐ Blocca l'app finché accetta
- ☐ Usa i sensori al posto del GPS

Domande di ripasso

Domanda 4 di 5

4. A cosa serve il Notification Channel su Android?

- ☐ A definire importanza e comportamento delle notifiche
- ☐ A installare Expo Go
- ☐ A compilare l'app iOS
- ☐ A creare un token Expo

Domande di ripasso

Domanda 5 di 5

5. Quando ha senso preferire Expo rispetto a RN CLI?

- ☐ Quando vuoi setup rapido e API pronte
- ☐ Quando vuoi massima configurazione nativa subito
- ☐ Quando non vuoi usare JavaScript
- ☐ Quando vuoi evitare qualsiasi build